

AI Runtime Governance: Vocabulary – Walkthrough Style

John M. Willis
Sustainable Future Tech
jwillis@sustainablefuturetech.com
[ORCID: 0009-0007-5889-9628](https://orcid.org/0009-0007-5889-9628)

Version 1.3
April 12, 2026

Abstract

This paper introduces a formalized execution-time vocabulary for AI runtime governance, establishing precise distinctions required to reason about control in autonomous and agentic systems. It defines a deterministic execution model in which systems do not directly execute actions, but instead propose state transitions that are evaluated against canonical state, compiled governance constraints, and derived authority to determine admissibility.

The paper clarifies that admissibility alone is insufficient for execution. At the commit boundary, binding validation re-derives authority, resolves ordering and interaction constraints, and deterministically selects which transitions are permitted to bind under the state that actually exists at that moment. Execution is therefore defined not as attempted action, but as successfully bound transition, establishing a precise boundary at which governance becomes real.

A core contribution is the disambiguation of commonly conflated concepts—including proposal versus execution, admissibility versus binding, authorization versus authority at commit, and evaluation versus enforcement—demonstrating that failure to maintain these distinctions leads to systems that appear governed but lack structural control.

The model further introduces the concept of an admissible set, recognizing that real systems often produce multiple valid transitions under concurrent conditions. Governance must therefore include deterministic adjudication at binding, rather than validation of a single candidate in isolation.

By defining governance as a function that resolves what becomes real at the point of state mutation, this work reframes AI governance from policy interpretation and post-hoc audit toward execution-time control. It provides a foundational vocabulary for aligning architectural implementations, enabling reproducible reasoning, and supporting the emergence of interoperable governance control planes across heterogeneous systems.

Keywords: AI Governance, Runtime Governance, Execution-Time Control, Admissibility, Commit Boundary, State Transition, Governance Control Plane, Binding Semantics, Canonical State, Deterministic Execution, Authority Re-Derivation, Agentic AI, Autonomous Systems, Runtime Enforcement, Auditability

Cite as: Willis, J. M. (2026). *AI Runtime Governance: Vocabulary – Walkthrough Style*. Zenodo.
<https://doi.org/10.5281/zenodo.19545394>

Execution Model (Summary)

A system does not execute actions directly.

It proposes transitions.

Those transitions are evaluated against canonical state using compiled governance and derived authority to determine admissibility.

This evaluation may produce not a single valid transition, but a set of admissible transitions. This is the normal case in concurrent systems, not an edge condition.

Admissibility alone is not sufficient for execution.

At the commit boundary, the system performs binding validation:

- authority is re-derived from current state
- ordering and interaction are resolved
- and the admissible set is deterministically resolved

Only transitions that satisfy these conditions under the state that actually exists at that moment are allowed to bind.

Binding is the point at which a transition becomes real.

Execution is therefore defined as a successfully bound transition—not an attempted action.

Each bound transition produces a new canonical state, which becomes the basis for all subsequent evaluation.

Vocabulary Agreement (Required Distinctions)

The following terms are not interchangeable and must be used precisely:

- **Proposed Transition** is not execution
- **Admissibility** is not binding
- **Authorization** is not authority at commit
- **Evaluation** is not enforcement
- **Admissible Set** is not a single decision
- **Governance Compilation** is not a system invariant

Each refers to a distinct step or constraint in the execution model.

If these distinctions are not maintained, both system design and analysis will be incorrect.

If these distinctions collapse, governance collapses into approximation.

Governance Roles and Ownership Domains

In a governance control plane architecture, ownership is not singular. It is distributed across distinct domains, each responsible for a different aspect of system behavior and control.

Clear separation of these roles is required to avoid ambiguity in accountability and to ensure that governance is both enforceable and auditable.

Policy Intent Owner

Defines governance intent (what should be allowed).
Examples include compliance, legal, and enterprise security functions.
This role produces policy but does not enforce it.

Governance Compiler Owner

Translates policy intent into executable constraints.
Produces structured artifacts such as schemas, rules, and constraint sets.
Determines how policy becomes machine-interpretable and enforceable.

Control Plane Owner

Operates the governance control plane.
Responsible for deterministic evaluation, invariant enforcement, and commit gating.
This is the runtime enforcement authority.

Execution Target Owner

Owns the system that performs the state change.
Applies authorized transitions but does not determine whether they are allowed.

Identity Authority (Human and Non-Human)

Establishes identity and credentials.
Provides authentication and base authorization context, but does not determine execution-time validity.

Commit Authority (Derived at Runtime)

The authority to allow a transition to bind under current conditions.
Not stored or assigned statically—derived at runtime from:

- identity
- policy
- constraints
- invariants
- canonical state

Authorization vs Authority

Authorization defines what an identity is permitted to do in general.

Authority is the proof that a specific action is permitted **now**, under the current system state and constraints.

A system may have:

- valid identity
- valid permissions
- correct policy evaluation

...and still produce an invalid outcome if authority is not re-derived at the moment of binding.

Key Principle

Governance is not owned by a single role.

It is enforced across layers and only becomes real at the point where a transition is allowed to bind.

Failure Mode Without Separation

If these ownership domains are not clearly defined:

- policy is mistaken for enforcement
- execution systems are assumed to provide governance
- identity is treated as sufficient for authority

This results in systems that appear governed but are not actually controlled at execution.

Detailed Walkthrough of Vocabulary

The sequence: from proposal to reality

1. A system does not act first. It proposes.

The sequence begins when an agent, service, workflow, or operator produces a **proposed transition**.

That proposed transition is not yet execution.

It is a candidate mutation of system state.

This matters because most people still talk as if the system is already “taking an action” at this point. It is not. It is only proposing that something change.

Examples:

- an agent proposes sending an email
- a CI/CD workflow proposes deploying a service
- an orchestration layer proposes changing access rights
- an AI workflow proposes writing to a database

At this stage, nothing has become real.

The proposed transition is just the system saying:

“Given what I think I know, this is what should happen next.”

That is the beginning of governance.

Not after execution. Before reality changes.

2. Governance is not invented here. It is compiled before this point.

Before a proposal can be evaluated, the system must already have a machine-evaluable governance model.

This is where **governance compilation** comes in.

Governance compilation is the translation of human governance intent into a form the system can actually evaluate.

This can include:

- domain-specific rules
- agent-specific constraints
- delegation models
- role relationships
- approval requirements
- domain-level invariants
- admissibility conditions

This is contextual.

It is not the same everywhere.

A finance agent and an infrastructure agent do not carry the same compiled governance.

So, this layer is:

- domain-specific
- agent-specific
- versioned
- contextual

But it is still not enforcement.

It is the preparation of the rules that will later be enforced.

This is also where one of the biggest confusions must be removed:

system invariants are not the same as **compiled governance**.

System invariants are structural and global.

Governance compilation is contextual and local.

For example:

- “ledger must be append-only” is a system invariant
- “this agent may not approve payments above X” is a compiled domain rule

The first defines what the system cannot violate.

The second defines what is allowed within the system.

That distinction has to remain clean.

3. The proposal is evaluated against canonical state, not against intention

Now the system has two things:

- a proposed transition
- a compiled governance model relevant to that proposal

But it still needs the third thing that actually matters:

canonical state

Canonical state is the authoritative representation of the system at that point in time.

This is the state the control plane is willing to treat as real for purposes of evaluation.

This is where many commenters have been pushing correctly: if the state is stale, partial, inconsistent, or ungrounded, then the system may still evaluate perfectly and still be wrong.

That is true.

But the control plane does not solve truth in the abstract.

What it does is require that the state it evaluates against be sufficiently qualified for use.

That means the state may be tested for things like:

- freshness
- completeness
- consistency
- provenance
- confidence threshold
- admissibility of the underlying record

So, at this stage, the system is not asking:

“What did the agent want?”

It is asking:

“What is the current canonical state against which this transition must be judged?”

This is the shift from reasoning quality to execution control.

4. Authorization is established upstream, but not trusted yet

This is another place people get confused.

Authorization is often treated as if it is granted once and then carried forward.

That is exactly the wrong model for autonomous systems.

Upstream, authorization may be established in the sense that the system can determine:

- who the actor is
- what delegation chain exists
- what permissions appear to apply
- what approvals have already been gathered
- what policy scopes are relevant

This is enough to evaluate admissibility.

But it is not enough to commit.

So the right phrasing is:

Authorization is established upstream,
but it is only valid if it can be re-derived at binding.

That means upstream authorization contributes to admissibility, but it is not final authority.

Nothing is final until binding.

5. Admissibility is evaluated

Now the control plane / substrate can perform the **admissibility function**.

Admissibility is not a feeling, a heuristic, or a policy slogan.

It is a deterministic function over:

- the current canonical state
- the proposed transition
- derived authority
- compiled governance
- system invariants

At this stage the system asks:

Is this proposed transition valid relative to the state, authority, and constraints now in force?

This is where domain-level invariants from compilation matter.

This is also where structural system invariants must still hold.

If the transition violates either:

- compiled constraints for that domain/agent
- or global system invariants

then it is not admissible.

What matters here is that the function is deterministic.

Same state.

Same proposal.

Same compiled governance.

Same authority.

Same invariants.

Then the same admissibility result must occur.

Otherwise replay cannot prove anything.

6. The result is not necessarily one valid transition. It may be an admissible set.

This is where an important refinement from recent discussions becomes essential.

Real systems often do not produce a single valid next move.

They can produce a **set of admissible transitions**.

That means multiple proposals may all be valid under:

- the same canonical state
- the same ordering snapshot
- the same compiled rules
- the same authority context

So, governance is not complete when the system merely says:

“These transitions are admissible.”

It must still determine:

“From this admissible set, which one or more are allowed to become real?”

This is not a separate architectural layer.

But it is a distinct responsibility that must be articulated clearly.

And this is the point that needs repeated explanation until it becomes standard language:

bind must be understood to include adjudication over an admissible set

Not just validation of one candidate in isolation.

7. Binding validation happens at commit

Now we arrive at the decisive moment.

This is the **commit boundary**.

This is where governance stops being descriptive and becomes operative.

Binding validation asks:

- does this transition still hold under the current canonical state?
- does the authority still hold now?
- has another transition already committed first?
- do ordering and interaction still permit this transition?
- from the admissible set, which one or more can actually bind?

This is the point where concurrency, interaction, and state drift matter most.

A transition that was admissible five milliseconds ago may no longer be admissible now.

Another transition may have already changed the state.

Delegation may have expired.

A revocation may have been issued.

A quota may have been consumed.

A dependency may have moved the world.

So binding validation is where the system re-derives and re-resolves everything against the state it actually encounters.

Not the state that existed when the proposal was first formed.

Not the state that existed when it was first evaluated.

The state that exists now.

This is why the clean statement is:

Authorization is not carried forward.

It is proven at commit.

And this is also where the other important statement belongs:

Admissibility defines the allowable set.

Binding validation resolves which one or more can actually commit.

8. Binding is the moment one or more admissible transitions become real

If binding validation succeeds, then binding occurs.

Binding is not the same thing as evaluation.

Binding is the point where a transition mutates canonical state.

This is where proposal ends and execution begins.

Execution should be defined very carefully:

Execution is not what the system attempted.
Execution is what successfully bound and committed.

That means if a transition is proposed, evaluated, even approved upstream, but does not bind, then it never became real.

This is the moment where governance either holds or disappears.

Because once the transition binds:

- state changes
- consequences begin
- external effects may occur
- reversibility may be lost

Everything before binding is governance preparation.

Binding is governance made real.

9. The new state is produced

Once a transition binds, the system moves into a **new canonical state**.

This new state is not just the old state plus an action.

It is the new baseline against which all future proposals must be evaluated.

This is why governance is recursive.

Each committed transition changes the state that determines the admissibility of every subsequent transition.

That is also why people perceive “trajectory” concerns.

But the system does not need to predict every possible future path.

It only needs to ensure that each bound transition produces a state that remains governable for whatever comes next.

That is the difference between:

- explicit long-range trajectory planning
- versus deterministic stepwise governance

The control plane constrains system evolution one committed step at a time.

See section below on execution target non-guarantee.

10. Auditability comes after binding, but depends on what was captured at binding

After commit, artifacts matter.

These can include:

- authorization artifact

- proof or attestation
- ledger entry
- state hash
- ordering record
- provenance chain
- execution receipt

These artifacts are not the decision itself.

They are representations of the decision.

That distinction is important.

But they are essential because without them:

- replay is impossible
- audit becomes narrative instead of proof
- the reason for the commit is lost
- trust becomes unverifiable

So, the clean statement is:

The boundary resolves admissibility in the moment.

Binding makes that resolution durable beyond the moment.

That is how governance becomes both correct and reconstructable.

Boundary of the Execution Model (Non-Guarantees)

The execution model defines what must be true for a transition to bind.

It guarantees:

- validity at the commit boundary
- enforcement of policy and invariants
- deterministic evaluation against canonical state

It does **not guarantee**:

- that the execution target correctly applies the transition
- that external systems behave as expected
- that human operators perform actions as intended

These sit outside the control-plane boundary.

The model assumes that once a transition is authorized and bound, the execution function $T(S, A)$ is applied correctly.

Failures in execution targets are not governance failures.
They are **execution-layer failures**.

What this sequence disambiguates

Admissibility is not binding

Admissibility means a transition is valid in principle under current conditions.
Binding means it is the transition that actually becomes real.

Authorization is not final upstream

Authorization may be established upstream.
It must be re-derived and proven at binding.

Governance compilation is not a system invariant

Compilation can include domain-level invariants and contextual rules.
System invariants are structural and globally enforced.

Multiple admissible transitions are normal

The system often has a set, not a singleton.
Binding validation must adjudicate over that set.

Execution is not intent

Execution is successful bind and commit.